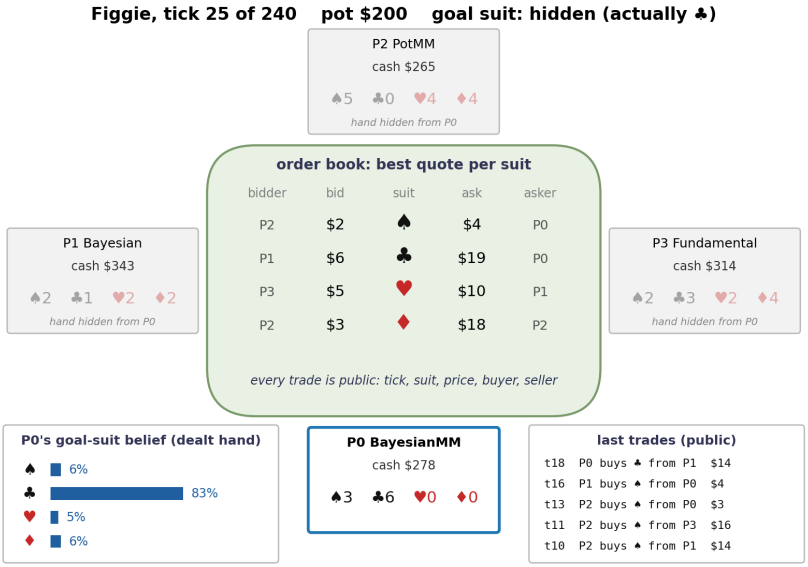
Reading the Order Flow: Information Extraction and Leakage in Learned Agents for the Trading Game Figgie

Nicholas Erup Larsen
IIIS, Tsinghua University
nglsls26@mails.tsinghua.edu.cn
Student ID: 2026400695

Adriaan Van der Berg
IIIS, Tsinghua University
adla26@mails.tsinghua.edu.cn
Student ID: 2026403362



1 Problem and motivation

In Figgie, 40 custom cards with no numbers or faces are dealt from one of 12 equally likely decks: one suit has 12 cards, one has 8 and two have 10. The goal suit is the **same color partner** of the 12-card suit (♠↔♣, ♥↔♦), i.e. if the deck is 12 ♠ / 10 ♣ / 10 ♥ / 8 ♦, the goal suit is ♣, and each ♣ pays $10 plus a pot bonus to whoever holds the most. Players trade continuously, so a player who eagerly buys ♣ is likely to hold many clubs and believe ♣ is the goal. Existing approaches, such as `figment`'s Bayesian market maker, `Figgie-bot`'s exact inference, and the `CS224R PPO` agent [1], compute or approximate the goal-suit posterior given only their own hand. Instead, we seek to answer three questions:

1. How much does the public trade history **reveal** about the goal suit beyond a single hand, and how quickly over a 240-tick game?
2. Do agents with memory (GRU, transformer) **learn to infer** the goal suit despite only receiving rewards for profit?
3. How much does each agent's own action stream reveal or **leak** about its hand, and do the leakiest agents lose the most to a learned best response (exploiter)?

Elo and profit against a fixed field are the only metrics reported for Figgie agents, and they say nothing about **why** an agent wins, or how robust it is. Figgie is a clean testbed for a central open question in reinforcement learning under partial observability: does an agent trained only on reward learn to infer hidden state, and to infer it from *other agents' behaviour*? Strong agents for imperfect-information games such as DeepStack [12] and Pluribus [8] sidestep the question by computing beliefs explicitly; model-free agents with memory are expected to learn them implicitly, but this is rarely checked, because in poker or StarCraft the exact belief is unknown or intractable. In Figgie it is exact: the 12-deck posterior from any set of cards is a closed-form sum, and the simulator records the true deck and every hand. Figgie also has the multi-agent twist that every

Preprint. Under review.

action is a signal: an agent that learns to read others' trades should also learn that its own trades are read, and exploitability, the standard robustness measure, has no closed form in a four-player general-sum game. The study exercises the core of the course: recurrent and transformer sequence models, representation learning via probes, and reinforcement learning with self-play.

2 Data

We use no external dataset: all data is generated by simulating Figgie in our own engine, so the amount is limited only by compute. The engine implements the standard rules and the CS224R action space [1], and runs four public agent families [2, 3, 4] whose decisions match their original code.

Quantity	Value
One game	240 ticks $\times$ 4 players $=$ 960 decisions
Observation per decision	69 `float32` $=$ 276 B
Training sample (obs., action, reward, value, log-prob, done)	$\approx$ 300 B
Full game log (deal, every quote and trade)	$\approx$ 10 KB
Simulator, measured (1 CPU core, pure Python)	11 ms/game $=$ 85,000 decisions/s
PPO training, measured (current pipeline)	3,800 learner steps/s $\approx$ 1.1 MB/s
Longest run so far	33 M learner steps $\approx$ 10 GB in 2.4 h
Projected: vectorised engine on a 32-core RTX 5090 server	$\sim$1 M steps/s $\approx$ 0.3 GB/s $\approx$ 1 TB/h

The current bottleneck is the Python simulator and opponents, not the network: a GRU-128 policy is tiny for any modern GPU. Porting the engine to C or batched GPU code, as we did for smaller imperfect-information games in prior work (about $150\times$ per GPU over one CPU core), is what moves throughput from the measured to the projected row. Because every game is logged with the true deck and all hands, each sample comes with exact ground truth for the belief and leakage measurements at no extra cost.

3 Related work and relevance to the field

Market microstructure. Kyle [6] and Glosten–Milgrom [7] model how informed traders' orders reveal private information and how market makers protect themselves against it. Those models assume rational agents with known strategies. We measure revelation and leakage for learned and heuristic agents in a market where ground truth is exact, which is impossible on real exchanges.

Multi-agent learning. Exploitability is the standard robustness measure in two-player zero-sum games, but it has no closed form in multi-player general-sum games such as Figgie or six-player poker [8]. Learned best responses [9] are the practical substitute; we pair them with an explanation of *why* an agent is exploitable.

Interpretability of RL agents. Probing has shown that AlphaZero encodes human chess concepts [10], using linear probes [11]. Figgie offers an exact Bayesian target for the probe, so we can say not only whether a belief is encoded but how close it is to optimal.

4 Preliminary approach

4.1 Baselines

Belief: the exact hand-only Bayesian posterior over the 12 decks, which every published agent approximates. **Agents:** the CS224R PPO agent and the Bayesian market makers from figment and Figgie-bot, all replicated and verified on our engine; these are the reference points for strength, leakage and exploitability.

4.2 Method

Revelation bound. Train a supervised transformer that reads the public trade history up to tick t and predicts the goal suit. Compare its accuracy and log-loss over t with the exact hand-only posterior. The gap is an empirical upper bound on what any agent can gain from the order flow, measured separately against each population of opponents.

Learned belief and mechanistic interpretability. We train PPO agents whose observation includes the trade history, comparing a GRU, a transformer and hand-crafted belief features, and open them up in three steps. Unlike interpretability studies whose targets are human-labelled concepts [10, 13], ours has an exact target: the Bayes-optimal goal-suit belief. We can therefore ask not only whether a belief is represented but whether the network computes the right one.

1. **Linear probing (what is represented).** Fit linear probes [11] from hidden states, at every tick and across training, to the exact hand-only posterior, the true goal suit, and each opponent's holding of each suit. A probe that beats the hand-only posterior on the true goal suit shows the agent has learned to read the order flow. Probes on a randomly initialised network of the same shape control for what the architecture alone makes decodable [14].
2. **Attribution (where the belief comes from).** For the transformer, find attention heads that attend to other players' purchases of a suit ("demand detectors"); for the GRU, attribute the goal-suit probe to individual past trades with gradient×input.
3. **Causal interventions (whether the agent uses it).** *Steering*: add the probe's "♣ is the goal" direction to the hidden state [15] and test whether the agent starts bidding up ♣. *Activation patching* [16]: swap the hidden state between two matched games that differ only in whether an opponent bought ♣, and test whether behaviour follows the patched state. *Ablation*: project the goal-suit direction out of the hidden state and measure the profit lost, a direct estimate of what the learned belief is worth.

As a stretch goal, we patch components one at a time in the small transformer to isolate the circuit [17] that carries opponent demand to goal-suit belief to bid.

Leakage. For each agent, train a classifier that predicts that agent's hand composition from its actions alone; its accuracy above the prior is the agent's leakage.

Exploitability. For each agent, train a PPO best response at one seat with the other seats fixed, and estimate exploitability as the best response's profit gain over a no-information control. We decompose the gain into an information term (earlier correct goal-suit belief) and a pricing term, and test whether leakage predicts exploitability across agents. If time allows, we validate the estimator on small games where exact exploitability is computable (Kuhn and Leduc poker).

Expected results. (i) Goal-suit accuracy versus tick for hand-only, order-flow and probed agent beliefs; (ii) a table of Elo, profit, leakage and estimated exploitability for every agent; (iii) whether learned memory architectures close the gap to the revelation bound.

4.3 What we measure

Quantity	Definition
Revelation gap $\Delta_{\text{rev}}(t)$	Log-loss of the exact hand-only goal-suit posterior minus log-loss of the best predictor that also sees the public trade history up to tick t, in bits; reported for each opponent population.
Belief extraction $E_a(t)$	Log-loss of a probe on agent a's hidden state for the true goal suit, placed between the hand-only posterior ($E = 0$) and the revelation bound ($E = 1$).
Leakage L_a	Mutual information, in bits, between agent a's hand composition and its action stream, lower-bounded by the cross-entropy gain of a trained classifier over the prior.
Exploitability $\hat{\varepsilon}_a$	Profit per game of a learned best response at one seat against three copies of a, minus that of a no-information control. Decomposed into an information term and a pricing term.
Strength	Mean profit per game with 95% CIs, win rate and Elo over rotated mixed tables (already measured in the replication).

The central tests are: does $E_a > 0$ for any learned agent; does E_a increase with history-aware architectures (transformer $>$ GRU $>$ none); and is the rank correlation between L_a and $\hat{\varepsilon}_a$ positive across agents.

4.4 What is new

- Prior Figgie work ranks agents only by profit or Elo against a fixed field: hand-written strategies in a discrete-event simulation [5], Bayesian and evolutionary bots [2, 3], and one PPO agent [1]. None measures what the order flow reveals or what an agent leaks.
- No published Figgie agent conditions its belief on opponents' trades; all use the player's own hand. We test whether learning closes that gap.
- No exploitability estimate exists for any Figgie agent, and none for a multi-player trading game that we are aware of.
- We release a single engine with four replicated sources whose decisions are verified against the original code, so future work can compare against the same agents.

Timeline. Midterm (3 weeks): replication results, revelation bound, first probe of PPO. Final (8 further weeks): history-aware PPO variants, leakage measurement, best-response exploitability and the leakage–exploitability analysis.

References

[1] D. Li Yang. Reinforcement learning for Figgie: learning negotiation as a first-class skill. Stanford CS224R project report, 2026.

[2] mneha05. figment: a self-play market-making arena for Figgie-style card markets. `https://github.com/mneha05/figment`.

[3] revilo-25. Figgie-bot. `https://github.com/revilo-25/Figgie-bot`.

[4] 0xDub. figgie-auto. `https://github.com/0xDub/figgie-auto`.

[5] C. DiSilvio, A. Luo, and A. Ozerov. Traders in a strange land: agent-based discrete-event market simulation of the Figgie card game. arXiv:2110.00879, 2021.

[6] A. S. Kyle. Continuous auctions and insider trading. *Econometrica*, 53(6), 1985.

[7] L. R. Glosten and P. R. Milgrom. Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *J. Financial Economics*, 14(1), 1985.

[8] N. Brown and T. Sandholm. Superhuman AI for multiplayer poker. *Science*, 365(6456), 2019.

[9] F. Timbers et al. Approximate exploitability: learning a best response. In *IJCAI*, 2022.

[10] T. McGrath et al. Acquisition of chess knowledge in AlphaZero. *PNAS*, 119(47), 2022.

[11] G. Alain and Y. Bengio. Understanding intermediate layers using linear classifier probes. arXiv:1610.01644, 2016.

[12] M. Moravčík et al. DeepStack: expert-level artificial intelligence in heads-up no-limit poker. *Science*, 356(6337), 2017.

[13] K. Li, A. K. Hopkins, D. Bau, F. Viégas, H. Pfister, and M. Wattenberg. Emergent world representations: exploring a sequence model trained on a synthetic task. In *ICLR*, 2023.

[14] J. Hewitt and P. Liang. Designing and interpreting probes with control tasks. In *EMNLP*, 2019.

[15] A. M. Turner et al. Steering language models with activation engineering. arXiv:2308.10248, 2023.

[16] K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in GPT. In *NeurIPS*, 2022.

[17] C. Olah, N. Cammarata, L. Schubert, G. Goh, M. Petrov, and S. Carter. Zoom in: an introduction to circuits. *Distill*, 2020.